| | |
|---|---|
| **Status** | Finished |
| **Started** | Monday, 28 September 2026, 12:17 PM |
| **Completed** | Monday, 28 September 2026, 12:40 PM |
| **Duration** | 22 mins 7 secs |
| **Grade** | **9.00** out of 10.00 (**90**%) |

Question **1**

Correct

Mark 1.00 out of 1.00

A sentence is a list of words that are separated by a single space with no leading or trailing spaces. Each word consists of lowercase and uppercase English letters.

A sentence can be shuffled by appending the 1-indexed word position to each word then rearranging the words in the sentence.

For example, the sentence "This is a sentence" can be shuffled as "sentence4 a3 is2 This1" or "is2 sentence4 This1 a3".

Given a shuffled sentence s containing no more than 9 words, reconstruct and return the original sentence.

Example 1:

**Input**:

is2 sentence4 This1 a3

**Output**:

This is a sentence

Explanation: Sort the words in s to their original positions "This1 is2 a3 sentence4", then remove the numbers.

Example 2:

**Input**:

Myself2 Me1 I4 and3

**Output**:

Me Myself and I

Explanation: Sort the words in s to their original positions "Me1 Myself2 and3 I4", then remove the numbers.

Constraints:

2 <= s.length <= 200

s consists of lowercase and uppercase English letters, spaces, and digits from 1 to 9.

The number of words in s is between 1 and 9.

The words in s are separated by a single space.

s contains no leading or trailing spaces.

**Answer:** (penalty regime: 0 %)

```
import sys


def reconstruct_sentence(s: str) -> str:
    # Store words using a dictionary: key = position, value = word text
    words = s.split()
    word_dict = {}

    for word in words:
        pos = int(word[-1])  # Extract the last character as the 1-indexed key
        text = word[:-1]  # Extract the word without the trailing digit
        word_dict[pos] = text

    # Sort keys from 1 to N and join the words into a sentence
    sorted_sentence = " ".join(word_dict[i] for i in sorted(word_dict.keys()))
    return sorted_sentence


if __name__ == "__main__":
    # Read input from standard input (handles single or multi-line grading inputs)
    input_data = sys.stdin.read().strip()
    if input_data:
        # Process line by line if multiple test cases exist
        for line in input_data.splitlines():
            if line.strip():
                print(reconstruct_sentence(line.strip()))
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | is2 sentence4 This1 a3 | This is a sentence | This is a sentence | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a number, convert it into corresponding alphabet.

| Input | Output |
|---|---|
| 1 | A |
| 26 | Z |
| 27 | AA |
| 676 | YZ |

**Input Format**

Input is an integer

**Output Format**

Print the alphabets

**Constraints**

1 <= num <= 4294967295

**Sample Input 1**

26

**Sample Output 1**

Z

**For example:**

| Test | Result |
|---|---|
| `print(excelNumber(26))` | `Z` |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def excelNumber(num: int) -> str:
    result = []
    while num > 0:
        # Subtract 1 because letters are 1-indexed (1 -> A, 26 -> Z)
        num -= 1
        # Get the remainder corresponding to the letter (0 -> 'A', 25 -> 'Z')
        remainder = num % 26
        result.append(chr(ord("A") + remainder))
        # Move to the next digit/column position
        num //= 26

    # Reverse the result list to get the correct order
    return "".join(reversed(result))


# Read input from standard input if required by automated platforms
if __name__ == "__main__":
    import sys

    input_data = sys.stdin.read().strip()
    if input_data:
        for line in input_data.splitlines():
            if line.strip():
                print(excelNumber(int(line.strip())))
```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(excelNumber(26))` | Z | Z | ✓ |
| ✓ | `print(excelNumber(27))` | AA | AA | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Give a dictionary with value lists, sort the keys by summation of values in value list.

**Input** : test_dict = {'Gfg' : [6, 7, 4], 'best' : [7, 6, 5]}

**Output** : {'Gfg': 17, 'best': 18}

**Explanation** : Sorted by sum, and replaced.

**Input** : test_dict = {'Gfg' : [8,8], 'best' : [5,5]}

**Output** : {'best': 10, 'Gfg': 16}

**Explanation** : Sorted by sum, and replaced.

Sample Input:

2

Gfg 6 7 4

Best 7 6 5

Sample Output

Gfg 17

Best 18

**For example:**

| Input | Result |
|---|---|
| 2<br>Gfg 6 7 4<br>Best 7 6 5 | Gfg 17<br>Best 18 |

**Answer:** (penalty regime: 0 %)

```
import sys


def sort_dict_by_value_sum():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Read number of key-value pairs
    n = int(input_data[0].strip())

    dict_sums = {}

    # Read N lines of dictionary data
    for i in range(1, n + 1):
        parts = input_data[i].split()
        key = parts[0]
        # Sum all remaining elements converted to integers
        val_sum = sum(int(x) for x in parts[1:])
        dict_sums[key] = val_sum

    # Sort dictionary by value sum in ascending order
    # dict.items() maintains insertion order of Python 3.7+ after sorting
    sorted_items = sorted(dict_sums.items(), key=lambda item: item[1])

    # Print each key and its total sum line by line
    for key, total_sum in sorted_items:
        print(f"{key} {total_sum}")


if __name__ == "__main__":
    sort_dict_by_value_sum()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 2<br>Gfg 6 7 4<br>Best 7 6 5 | Gfg 17<br>Best 18 | Gfg 17<br>Best 18 | ✓ |
| ✓ | 2<br>Gfg 6 6<br>Best 5 5 | Best 10<br>Gfg 12 | Best 10<br>Gfg 12 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

In the game of Scrabble™, each letter has points associated with it. The total score of a word is the sum of the scores of its letters. More common letters are worth fewer points while less common letters are worth more points. The points associated with each letter are shown below:

Points Letters

1 A, E, I, L, N, O, R, S, T and U

2 D and G

3 B, C, M and P

4 F, H, V, W and Y

5 K

8 J and X

10 Q and Z

Write a program that computes and displays the Scrabble™ score for a word. Create a dictionary that maps from letters to point values. Then use the dictionary to compute the score.

A Scrabble™ board includes some squares that multiply the value of a letter or the value of an entire word. We will ignore these squares in this exercise.

Sample Input

REC

Sample Output

REC is worth 5 points.

**For example:**

| Input | Result |
|---|---|
| REC | REC is worth 5 points. |

**Answer:** (penalty regime: 0 %)

```
import sys


def compute_scrabble_score():
    # Mapping each uppercase letter to its point value
    score_map = {
        'A': 1,
        'E': 1,
        'I': 1,
        'L': 1,
        'N': 1,
        'O': 1,
        'R': 1,
        'S': 1,
        'T': 1,
        'U': 1,
        'D': 2,
        'G': 2,
        'B': 3,
        'C': 3,
        'M': 3,
        'P': 3,
        'F': 4,
        'H': 4,
        'V': 4,
        'W': 4,
        'Y': 4,
        'K': 5,
        'J': 8,
        'X': 8,
        'Q': 10,
        'Z': 10,
    }

    # Read input word
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    word = input_data.strip()

    # Calculate total score by looking up each character in upper case
    total_score = sum(score_map.get(char.upper(), 0) for char in word)

    # Print output matching the required sample format
```

```
    print(f"{word} is worth {total_score} points.")


if __name__ == "__main__":
    compute_scrabble_score()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | GOD | GOD is worth 5 points. | GOD is worth 5 points. | ✓ |
| ✓ | REC | REC is worth 5 points. | REC is worth 5 points. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Incorrect

Mark 0.00 out of 1.00

Create a student dictionary for n students with the student name as key and their test mark assignment mark and lab mark as values. Do the following computations and display the result.

1.Identify the student with the highest average score

2.Identify the student who as the highest Assignment marks

3.Identify the student with the Lowest lab marks

4.Identify the student with the lowest average score

Note:

If more than one student has the same score display all the student names

Sample input:

4

James 67 89 56

Lalith 89 45 45

Ram 89 89 89

Sita 70 70 70

Sample Output:

Ram

James Ram

Lalith

Lalith

**For example:**

| Input | Result |
|---|---|
| 4<br>James 67 89 56<br>Lalith 89 45 45<br>Ram 89 89 89<br>Sita 70 70 70 | Ram<br>James Ram<br>Lalith<br>Lalith |

**Answer:** (penalty regime: 0 %)

```
# Read the number of students
n = int(input())

students = {}
order_of_insertion = []

# Populate the dictionary
for _ in range(n):
    data = input().split()
    name = data[0]
    test_mark = float(data[1])
    assign_mark = float(data[2])
    lab_mark = float(data[3])

    # Calculate average score
    avg_score = (test_mark + assign_mark + lab_mark) / 3.0

    # Store marks and average in the dictionary
    students[name] = {
        'test': test_mark,
        'assign': assign_mark,
        'lab': lab_mark,
        'avg': avg_score
    }
    order_of_insertion.append(name)

# 1. Identify the student(s) with the highest average score
max_avg = max(s['avg'] for s in students.values())
highest_avg = [name for name in order_of_insertion if students[name]['avg'] == max_avg]

# 2. Identify the student(s) who has the highest Assignment marks
max_assign = max(s['assign'] for s in students.values())
highest_assign = [name for name in order_of_insertion if students[name]['assign'] == max_assign]
```

```
34
35 # 3. Identify the student(s) with the Lowest lab marks
36 min_lab = min(s['lab'] for s in students.values())
37 lowest_lab = [name for name in order_of_insertion if students[name]['lab'] == min_lab]
38
39 # 4. Identify the student(s) with the lowest average score
40 min_avg = min(s['avg'] for s in students.values())
41 lowest_avg = [name for name in order_of_insertion if students[name]['avg'] == min_avg]
42
43 # Print each result on a separate line as required by the test framework
44 print(" ".join(highest_avg))
45 print(" ".join(highest_assign))
46 print(" ".join(lowest_lab))
47 print(" ".join(lowest_avg))
48
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 4<br>James 67 89 56<br>Lalith 89 45 45<br>Ram 89 89 89<br>Sita 70 70 70 | Ram<br>James Ram<br>Lalith<br>Lalith | Ram<br>James Ram<br>Lalith<br>Lalith | ✓ |

Your code failed one or more hidden tests.

Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

You are given a string `word`. A letter is called **special** if it appears both in lowercase and uppercase in `word`.

Your task is to return the number of **special** letters in `word`.

## Constraints

- The input string `word` will contain only alphabetic characters (both lowercase and uppercase).
- The solution must utilize a dictionary to determine the number of special letters.
- The function should handle various edge cases, such as strings without any special letters, strings with only lowercase or uppercase letters, and mixed strings.

## Examples

**Example 1:**

Input: word = "aaAbcBC"
Output: 3

Explanation:
The special characters in `word` are 'a', 'b', and 'c'.

Example 2:

Input: word = "abc"
Output: 0

Explanation:
No character in `word` appears in uppercase.

**For example:**

| Test | Result |
|---|---|
| `print(count_special_letters("AaBbCcDdEe"))` | 5 |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def count_special_letters(word: str) -> int:
    # Dictionary to track case appearances for each letter
    # Structure: { 'a': {'has_lower': False, 'has_upper': False} }
    letter_dict = {}

    for char in word:
        lower_char = char.lower()

        # Initialize dictionary entry if character hasn't been seen yet
        if lower_char not in letter_dict:
            letter_dict[lower_char] = {"has_lower": False, "has_upper": False}

        # Update lower/upper status
        if char.islower():
            letter_dict[lower_char]["has_lower"] = True
        elif char.isupper():
            letter_dict[lower_char]["has_upper"] = True

    # Count letters where both lower and upper flags are True
    special_count = 0
    for status in letter_dict.values():
        if status["has_lower"] and status["has_upper"]:
            special_count += 1

    return special_count
```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(count_special_letters("AaBbCcDdEe"))` | 5 | 5 | ✓ |
| ✓ | `print(count_special_letters("ABCDE"))` | 0 | 0 | ✓ |
| ✓ | `print(count_special_letters("abcde"))` | 0 | 0 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

A sentence is a string of single-space separated words where each word consists only of lowercase letters.A word is uncommon if it appears exactly once in one of the sentences, and does not appear in the other sentence.

Given two sentences s1 and s2, return a list of all the uncommon words. You may return the answer in any order.

Example 1:

Input: s1 = "this apple is sweet", s2 = "this apple is sour"

Output: ["sweet","sour"]

Example 2:

Input: s1 = "apple apple", s2 = "banana"

Output: ["banana"]

Constraints:

1 <= s1.length, s2.length <= 200

s1 and s2 consist of lowercase English letters and spaces.

s1 and s2 do not have leading or trailing spaces.

All the words in s1 and s2 are separated by a single space.

Note:

Use dictionary to solve the problem

**For example:**

| Input | Result |
|---|---|
| this apple is sweet<br>this apple is sour | sweet sour |

**Answer:** (penalty regime: 0 %)

```
import sys


def uncommon_from_sentences(s1: str, s2: str) -> list:
    # Dictionary to store word counts across both sentences
    word_counts = {}

    # Combine both sentences into a single list of words
    all_words = s1.split() + s2.split()

    # Count occurrences of each word in the dictionary
    for word in all_words:
        word_counts[word] = word_counts.get(word, 0) + 1

    # An uncommon word appears exactly once total
    uncommon_words = [
        word for word, count in word_counts.items() if count == 1
    ]

    return uncommon_words


if __name__ == "__main__":
    # Read input lines cleanly
    lines = [
        line.strip() for line in sys.stdin.read().splitlines() if line.strip()
    ]
    if len(lines) >= 2:
        s1 = lines[0]
        s2 = lines[1]
        result = uncommon_from_sentences(s1, s2)
        # Print space-separated output matching sample test format
        print(*result)
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | this apple is sweet<br>this apple is sour | sweet sour | sweet sour | ✓ |
| ✓ | apple apple<br>banana | banana | banana | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program that takes an input string from the user and counts the number of occurrences of each vowel (a, e, i, o, u) in the string. The program should be case-insensitive, meaning it should treat uppercase and lowercase vowels as the same.

**Description:**

Vowels play a significant role in the English language and other alphabet-based languages. Counting vowels in a given string is a fundamental task that can be applied in various text processing applications, including speech recognition, linguistic research, and text analysis. The objective of this problem is to create a Python script that accurately counts and displays the number of times each vowel appears in a user-provided string.

**Program Requirements:**

**Input:**

First line reading String as input, The string can contain any characters, including letters, numbers, and special characters.

**Output:**

Display the number of occurrences of each vowel in the string.

The output should list each vowel followed by its count.

**Example:**

Consider the following example for better understanding:

- **Input:** "Python Programming"
- **Output**

  **a = 1**
  **e = 0**
  **i = 1**
  **o = 2**
  **u = 0**

**For example:**

| Input | Result |
|---|---|
| Hello World | a = 0<br>e = 1<br>i = 0<br>o = 2<br>u = 0 |
| Python | a = 0<br>e = 0<br>i = 0<br>o = 1<br>u = 0 |

**Answer:** (penalty regime: 0 %)

```
import sys


def count_vowels():
    # Read input string
    input_text = sys.stdin.read()
    if not input_text:
        return

    # Take the first line of input and convert to lowercase for case-insensitivity
    text = input_text.splitlines()[0].lower()

    # Initialize a dictionary for vowels in required alphabetical order
    vowel_counts = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}

    # Count occurrences of each vowel
    for char in text:
        if char in vowel_counts:
            vowel_counts[char] += 1

    # Display results in the required format: "vowel = count"
    for vowel, count in vowel_counts.items():
        print(f"{vowel} = {count}")


if __name__ == "__main__":
    count_vowels()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | Hello World | a = 0<br>e = 1<br>i = 0<br>o = 2<br>u = 0 | a = 0<br>e = 1<br>i = 0<br>o = 2<br>u = 0 | ✓ |
| ✓ | AEIOU aeio u | a = 2<br>e = 2<br>i = 2<br>o = 2<br>u = 2 | a = 2<br>e = 2<br>i = 2<br>o = 2<br>u = 2 | ✓ |
| ✓ | Python | a = 0<br>e = 0<br>i = 0<br>o = 1<br>u = 0 | a = 0<br>e = 0<br>i = 0<br>o = 1<br>u = 0 | ✓ |
| ✓ | abcdefghijklmnopqrstuvwxyz | a = 1<br>e = 1<br>i = 1<br>o = 1<br>u = 1 | a = 1<br>e = 1<br>i = 1<br>o = 1<br>u = 1 | ✓ |
| ✓ | 12345!@#$%AEIOU | a = 1<br>e = 1<br>i = 1<br>o = 1<br>u = 1 | a = 1<br>e = 1<br>i = 1<br>o = 1<br>u = 1 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given an array of names of candidates in an election. A candidate name in the array represents a vote cast to the candidate. Print the name of candidates received Max vote. If there is tie, print a lexicographically smaller name.

**Examples:**

Input : votes[] = {"john", "johnny", "jackie",

"johnny", "john", "jackie",

"jamie", "jamie", "john",

"johnny", "jamie", "johnny",

"john"};

Output : John

We have four Candidates with name as 'John', 'Johnny', 'jamie', 'jackie'. The candidates John and Johny get maximum votes. Since John is alphabetically smaller, we print it. Use dictionary to solve the above problem

**Sample Input:**

10

John

John

Johny

Jamie

Jamie

Johny

Jack

Johny

Johny

Jackie

**Sample Output:**

Johny

**Answer:** (penalty regime: 0 %)

```
import sys


def find_election_winner():
    # Read all lines from input
    lines = [
        line.strip() for line in sys.stdin.read().splitlines() if line.strip()
    ]
    if not lines:
        return

    # First line is the number of votes, followed by candidate names
    num_votes = int(lines[0])
    votes = lines[1 : num_votes + 1]

    # Use a dictionary to count votes for each candidate
    vote_counts = {}
    for candidate in votes:
        vote_counts[candidate] = vote_counts.get(candidate, 0) + 1

    # Find maximum votes received by any candidate
    max_votes = max(vote_counts.values())

```

```
    # Get all candidates who received the maximum number of votes
    top_candidates = [
        candidate
        for candidate, count in vote_counts.items()
        if count == max_votes
    ]

    # In case of a tie, select the lexicographically smaller candidate
    winner = min(top_candidates)

    print(winner)


if __name__ == "__main__":
    find_election_winner()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 10<br>John<br>John<br>Johny<br>Jamie<br>Jamie<br>Johny<br>Jack<br>Johny<br>Johny<br>Jackie | Johny | Johny | ✓ |
| ✓ | 6<br>Ida<br>Ida<br>Ida<br>Kiruba<br>Kiruba<br>Kiruba | Ida | Ida | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

A company wants to send its quotation secretly to its client. The company decided to encrypt the amount they are sending to their client with some special symbols so that the equation amount will not be revealed to any external person. They used the special symbols !,@,#,$,%,^,&,*,>,< for 0,1,2,3,4,5,6,7,8,9 respectively. Write a python code to help the company to convert the amount to special symbols.

(Value rounded off to 2 decimal points)

Input

n: Float data type which reads amount to send

Output

s: : String data type which displays symbols

Sample Testcase 1

Input

10000

Output

@!!!!.!!

Sample Testcase2

1234.56

Output

@#$%.^&

**For example:**

| Input | Result |
|---|---|
| 1345.23 | @$%^.#$ |
| 15000.59 | @^!!!.^< |
| 156789 | @^&*><.!! |

**Answer:** (penalty regime: 0 %)

```
import sys


def encrypt_amount():
    # Read input string
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Convert string to float and round off to 2 decimal places as required
    amount = float(input_data.splitlines()[0])
    formatted_amount = f"{amount:.2f}"

    # Symbol mapping dictionary: digit character -> symbol
    symbol_map = {
        '0': '!',
        '1': '@',
        '2': '#',
        '3': '$',
        '4': '%',
        '5': '^',
        '6': '&',
        '7': '*',
        '8': '>',
        '9': '<',
        '.': '.',  # Decimal point remains unchanged
    }

    # Convert each character of the formatted string using the dictionary
    encrypted_symbol = "".join(
        symbol_map.get(char, char) for char in formatted_amount
    )

    print(encrypted_symbol)


if __name__ == "__main__":
    encrypt_amount()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 1345.23 | @$%^.#$ | @$%^.#$ | ✓ |
| ✓ | 15000.59 | @^!!!.^< | @^!!!.^< | ✓ |
| ✓ | 1234 | @#$%.!! | @#$%.!! | ✓ |
| ✓ | 156789 | @^&*><.!! | @^&*><.!! | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.